10

Control

This chapter on control is the last chapter of this text concerning the design and build sequence of automation. Chapters 2–10 present the core material machine builders or equipment specification writers need to know on how the process can and should work. The remaining three chapters will add information with larger case studies of new automation products (Chapter 11), specifications (Chapter 12), and the larger group of packaging machines (Chapter 13).

Control is the part of automation from which many mechanical engineers shy away. The often-heard comment is, "let the electrical engineers do that." This comment and attitude are unfortunate, since with today's control options, almost any type of engineer can create a standard level of control implementation.

The concepts within control can be seen from the following situation. Imagine that you have had a bad week, made up of many bad days (perhaps as outlined in the beginning of Chapter 9). You are home on a Sunday afternoon, and you want to forget about your "wonderful" job of stacking boxes. You want to enjoy a mindless three hours of football on your television. After all, you deserve a break.

So on the table on your right you have a bowl of chips and a glass of your favorite cold beverage. On your left are the remote control and your cell phone. The remote is important since your two favorite teams are playing on two different channels at the same time, and you want to follow them both. The cell phone will let you call your friend who roots for one of the teams challenging one of

Copyright © 2005 by Marcel Dekker

your favorite teams. The television is in front of you, and you have ordered a pizza for delivery around half time. Any distractions (dog, kids, and spouse) are out shopping or whatever. Life seems under control. What do you have to do?

- Place more beverage in the refrigerator to be cold when you need it.
- Turn on the television.
- Lapse into an "eat a chip then take a sip" mode with your right hand.
- Keep the remote in your left hand to flip whenever the mood strikes or there is a commercial.
- Decide if your team is beating your friend's team enough so as to call and gloat.
- Listen to the doorbell for the pizza delivery person.

This does not seem like work, yet the decision process you are undergoing is probably more demanding than that of your job. This can be seen by a more detailed comparison of these fun tasks to an automatic machine.

Placing more beverages in the refrigerator represents the need to plan ahead. Similarly, you hopefully purchased enough chips the day before the game. If the supply of raw materials into an automatic machine goes to zero, then the machine will waste available production time. Some control programs monitor the production rate and help to inform the inventory control program so as to electronically order more raw materials from the supplier in a Just In Time (JIT) mode.

Your turning the television on emulates the controller signaling a required external device. In this example you only have to do it once, but if you somehow ignored this fact, the afternoon would be wasted watching a blank screen. Automation machine controllers can interface with almost any other device if needed. The number of devices and how they are to be signaled (the number of digital and analog controller output lines) will help determine the size and type of the controller and its options.

The "eat a chip then take a sip" mode is the normal operational mode of your afternoon. In this example, there is no penalty or problem if you happen to eat two chips before sipping, of if you grab more than one chip at a time. For most automation processes, this would not hold true. The order would be more explicit, but this does represent a normal controller process loop where, if the supplies hold true and there are no interruptions, the loop would continue forever. The loop gets broken if and when there is an interruption.

The flipping of stations with the remote is a second independent program loop that is not coupled to eating chips and sipping beverage. Whether or not you have just eaten a chip or taken a sip will not determine if and when the game production staff determines that a block of television commercials needs to be run. And no matter how hard you may scream for your teams, the timing of flipping between games cannot control the games' outcome. Automation

Copyright © 2005 by Marcel Dekker

controllers can handle more than one program loop, but as an engineer adds more loops to the controller's tasks, the time between successive executions of the same loop becomes longer.

The rest of the afternoon's tasks are interruptions. Some of these are good interruptions, like the door bell signaling that the pizza has arrived. And some could be bad, like when you are out of chips, beverage, a bathroom break, or your team is losing and your friend calls on the cell phone to gloat. The order and the magnitude of these interruptions need to be assessed. Most likely the door bell for the pizza will take precedence over refilling your glass. You do not want to have the pizza delivery person wait too long, or they might leave. Similarly in automation, one needs to plan out all of the parallel scenarios, such that an operator opening a door that trips a safety circuit immediately executes a machine shutdown, rather than worry about signaling the supplier to send more parts to be assembled. Safety has a higher precedence over the machine being out of supplies for 10 minutes more.

The cell phone also represents a device that can be used by you to send a good signal when the conditions so trigger (your team is up 14 points and it is time to gloat to your friend), and a device that receives the news that you do not want to hear. In automation, some interfaced devices work with two directional communications. These devices can be additional controllers or processors that are specifically meant to unburden the main controller.

So our enjoyable afternoon is actually a set of challenging tasks and interrupts handling situations. If it was not so much fun, it would be thought of as work! But to a machine controller, there is no fun or work, just a set of signals to accept, process, and send.

10.1. TIMING DIAGRAMS

When an automation process is supposed to occur can and should be documented in one of several forms. It is only when the order of steps and the related input and output signals have been laid out on paper or the computer monitor that the simplicity or complexity of a controller can be assessed. The example concerning football watching could be represented by a flowchart similar to a computer program, but it will be more instructive to look at a series of machines.

If one looks at the standard home washing machine, the traditional controller is a timer motor-driven set of cams that trigger the different devices. The timer motor does not turn as the washer is being filled with water. A pressure sensor confirms that the water level has been reached, and then the timer motor can proceed. The washer then agitates for a prescribed amount of time, followed by a pumping out of the dirty water, a rinse (with a small amount of water and a short agitation), and a spin. Some washers have additional operations, but they will be ignored for now.

Copyright © 2005 by Marcel Dekker

Figure 10.1 shows a timing diagram of these operations and the sensors or motors either triggering events or causing motion. The sensor input is considered "low" or 0 if there is no signal, or "high" or 1 when the signal occurs (as when the water level has been reached). The agitation motor is at rest when noted as 0, and in operation with a 1. (The same motor is used to both agitate and pump out water in real life, by operating the motor at two different speeds. But this will be ignored here.) There are several variations of timing diagrams used, but they all state the same information.

It should be noted that in Fig. 10.1 the *X*-axis is a nonlinear statement of time. Some events like "Turn On" take a fraction of a second, while other events like "Fill" can take five minutes or more. The "Drain" operation repeats during the "Spin" cycle so as to remove the water that has been wrung out of the clothes.

Historically, there is another answer, when there is relatively no input signal to be processed. A paper tape can be punched with a series of holes to demark when the various output devices should be actuated. The only variation a paper tape is capable of processing is that of stopping the tape until something else has occurred. This was also used in old-time player pianos (seen in cowboy western movies), where popular songs were encoded into many rows of holes in paper rolls often 8 in. in width. The rolls were set in motion, and then the holes would allow for compressed air to escape. When such an escape occurred, the corresponding key on the piano was played. The song would sound the same each time played, and

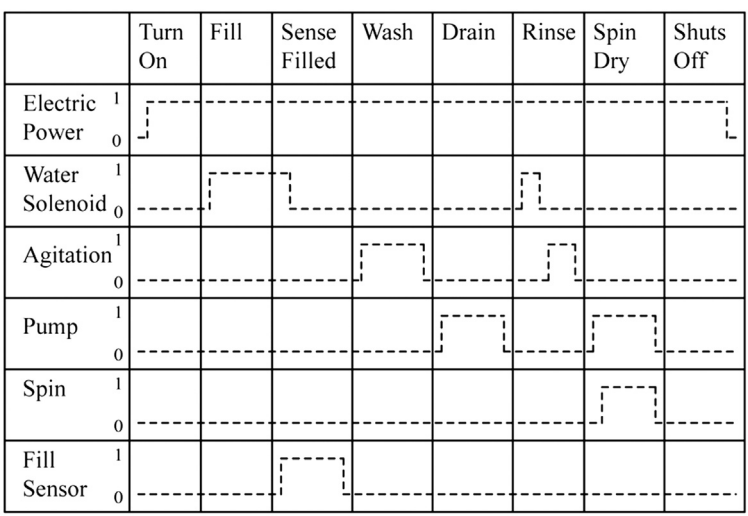


FIG. 10.1 Timing diagram for simple washing machine operation

Copyright © 2005 by Marcel Dekker

there are no options. The songs are hard coded for life unless one pokes an additional hole with a pencil (or a bullet in the old western movies).

The implementation of a timing diagram for most automation today can be done on a PC, a Programmable Logic Controller (PLC), or other dedicated processors. As mentioned before, most older standard washers do not use any of these options, but essentially wrap the output timing diagram curves each onto its own cam surface. Then each cam has a limit switch that follows the cam surface due to light spring forces. The input timing diagram signals will stop or start the timing motor and cams, as per the water level sensing operation. It is the cam limit switches that are the most likely element to fail in a washing machine, but since they are integrated into the cam and timer motor, are not inexpensive to replace!

Another option less likely found in automation but more likely found in consumer products is that of a programmable controller chip. An automatic breadmaking machine can be purchased for less than $100. It does operate an electric motor and turn on the heating coil. User-operated switches select options. It does not need nor can it afford a low end PLC costing $200. The controller chip used can and does only cost a few dollars at most. There is no requirement to be able to reprogram your breadmaker to do other variations (though it does sound like fun!) so the control code can be burned into the chip once and for all.

10.2. PROGRAMMABLE LOGIC CONTROLLERS

Many control applications require switching (on or off) of various outputs as a function of a number of inputs. This type of control is known as logic control or switching control. The relative simplicity of switching control makes it attractive for use in automatic machines where there is a requirement for the machine to follow a set sequence of operations. The use of switching control to cause a machine to go through a set sequence of operations gives rise to the term sequential control.

A PLC is an industrialized dedicated microcontroller whose operation is based primarily on sequential control of a process. Its Real Time Operating System (RTOS) scans a program in a repetitive cycle and actions are taken (outputs set) depending on logical relationships (specified by the programmer) between the inputs.

10.2.1. Are PLCs Necessary?

If one had an automatic machine with only a few motors, and one wanted to avoid controllers at all costs due to one's silly fear of electronics, one could control the machine with a bank of switches (Fig. 10.2). Now an operator would be needed to

Copyright © 2005 by Marcel Dekker

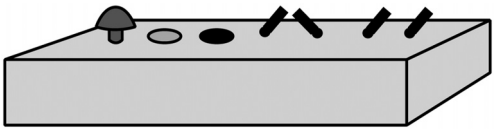


FIG. 10.2 Bank of switches

flip all of these switches when things looked appropriate, rather than the operator performing the task themselves. Now if it was the operation of a heavy capacity overhead crane, this makes all the sense in the world, but if it is a machine to tie bows, this borders on stupidity.

So the banks of switches needs to be replaced with some electric relays, which themselves are activated electronically by a controller. Now no matter what kind of controller is used (even paper tapes) the concept of a relay still prevails. The relay's electromagnet holding a metal arm making an electrical contact is usually replaced by a solid-state version. If it is to conduct a larger power source, a relay can now be a dedicated device several cubic centimeters in size. But if it is a computer-type 5 V TTL signal, a single IC chip could process 10 or 20 relays internally.

The physical use of these older electromechanical relays has diminished, but the concept of the relay lives on. So it is important to understand the older notation, not just for historical reasons, but as a basis of present-day understanding. Figure 10.3 shows the four basic elements, the relay coil, which acts as an electromagnet, the normally open and normally closed switches, which can get activated by the relay coil, and the lamp, which lights up to show that a circuit is closed. These elements are replaced by present-day solid-state devices, so one might not hear the click as one did when an electromagnetic relay closed.

These four elements do not facilitate any human or environmental interaction. Any On/Off or Emergency switches, or any sensor input, needs to be included. Figure 10.4 shows the representation of the six basic switch types. For example, a complex optical sensor should state somewhere (perhaps even on the sensor body itself) what type of switch it behaves like.

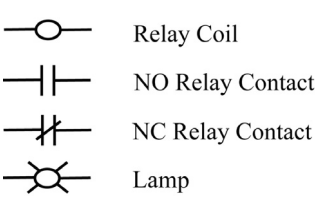


FIG. 10.3 PLC programming elements

Copyright © 2005 by Marcel Dekker

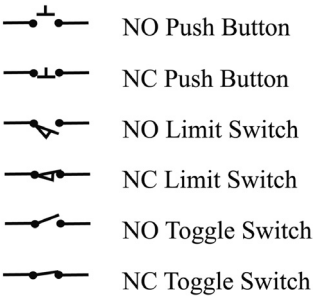


FIG. 10.4 PLC programming elements

Since all of these switches either close a circuit or open a circuit when they are activated, all of the switch types are reduced symbolically to the two representations in Fig. 10.5. They simply state the continuity of the local part of the circuit (allowing current to flow or not), and the engineer is left to understand the physical reality of the situation. This may sound more difficult, but when these symbols are used in larger programs, simplicity is a must.

10.2.2. Ladder Logic

A PLC, whether it is the size of a small brick or a desktop PC, will be programmed in one logic language or another. Most PLCs have several rows of connections for input and output wires. Programming is done with either a teach pendant containing a keyboard full of specific buttons and a simple LCD display, or by a laptop PC. The standard language for years has been called Ladder Logic. Ladder Logic is predicated on the extensive use of relays in the past. A symbol-based solution used to translate electrical schematics into program instructions the PLC can execute. There are standards (IEC 1131 is the modern standard) on what should and should not be in Ladder Logic, but many manufacturers do have some extensions that make their models more powerful than the competition. So beware if you have Brand A code and you hope to download it into Brand B PLC (or even Brand A model 2) without some changes.

One of the key concepts as one writes larger amounts of Ladder Logic in a single executable program is that of Scan Time. Scan Time is the time to make

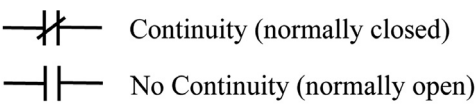


FIG. 10.5 Continuity

Copyright © 2005 by Marcel Dekker

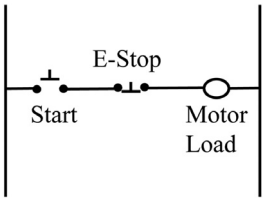


FIG. 10.6 Single rung starter motor circuit

one pass through the program, and one pass can be thought of as one PLC cycle. During each cycle, inputs are sampled, logic is evaluated, and outputs are set. The Scan Time is an indication of how often the logic functions are evaluated.

A written expression of a program using Ladder Logic is called a Ladder Diagram (LD). Here is where the term "ladder" makes sense. Each step in the program is written as a rung on a ladder. One starts at the top rung, and cycles downward, then returns to the top and repeats until ended or interrupted. There are a few general rules on what should or should not appear on a Ladder Diagram:

- Inputs may be repeated in a diagram.
- Only one output should occur in any rung.
- A specific output should appear only once in a diagram.

This will be understood best by stepping through a series of LD examples.

Figure 10.6 shows a single rung that is supposed to start an electric motor. Note the motor symbol is the same as a relay coil, something that causes action when a current is passed. The Start switch is a momentary NO switch that in real life is a 1 in. diameter green button. The NC E-Stop is usually a red mushroom button that is easy to identify on all machines. When the Start button is pressed, the current can flow from the left ladder side to the right ladder side, but the Start button does not stay depressed when one releases it. So the motor starts to run only as long as the button is pressed. This is not very useful in factory automation.

So, to make this a useful LD circuit, a load relay is added in parallel to the Start button (Fig. 10.7). When the Start button is pressed for a fraction of a second, the energy of the motor also closes a relay, which will keep the motor running after the Start button is released. The power to the load relay is disconnected when the E-Stop button is pressed for a fraction of a second. If the E-Stop button was not included in this LD logic, there would be no method of turning the motor off, short of disconnecting the power source.

This logic is how the older controller systems could have functioned, but it would not be convenient to add a load relay to every motor one wanted to control. So the same logic is applied to only internal PLC elements (Fig. 10.8). The Start and E-Stop switches have been updated to the continuity symbols, and the motor

Copyright © 2005 by Marcel Dekker

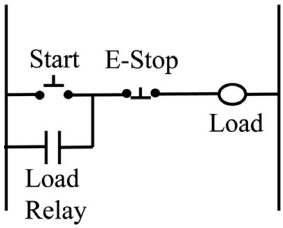


FIG. 10.7 Corrected single rung starter motor circuit

relay has been replaced by an internal relay, both the energized coil and the element in parallel to the Start switch. This is an easier method to implement and change the logic if need be. The only difference is that, without some sensor to confirm that the motor has started and continues to run, it could be possible for the PLC to think that the motor is running because the internal relay says it is, and perhaps due to a loose wire, the motor has stopped running an hour ago.

10.2.3. Model Train Example 1*

A model train can be configured as in Fig. 10.9. This could look like what one would set up on the living room floor, but if one has ever run a model train that has an inner and outer loop, one knows that not only does switch S1 need to be set, but switch S2 has to match it (inner or outer loop) or the train will derail. This may be fun to watch now and again, but it does mean one has to reset the train.

So a PLC can be set up to monitor the situation to guarantee that there will be no derailments. A possible scenario would be:

- The train should start when the NO start switch is pressed;
- The NC stop switch should stop the train;

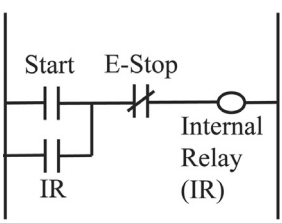


FIG. 10.8 Two-rung software ladder

- Train should only run when BOTH switches (S1 and S2) are in the same position.

Copyright © 2005 by Marcel Dekker

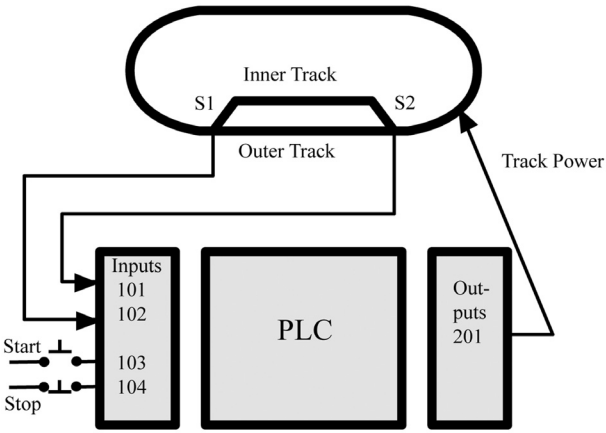


FIG. 10.9 Model train example 1

This means that for the few seconds one has changed one of the switches (S1 or S2) and not the other, the train power will be off. This is not exactly useful, but will be improved in the next example.

One of many possible Ladder Logic Diagrams is given in Fig. 10.10. The 100 numbers are reserved for inputs, the 200 numbers for outputs, and the 300 numbers for relays. The upper two rungs will function as the previous motor starter example, except that the powered up device is only the relay R301. Then in the lower two rungs, the output power to 201 only happens if the relay R301 is energized, and the two switches are in the same direction.

This example does seem a bit highly engineered for an application for a toy, but the logic it demonstrates is a good stepping stone. The author is not suggesting that one would want to implement this in real life.

10.2.4. Model Train Example 2*

To learn more about Ladder Logic, more constraints will be added to the model train layout. Now suppose the following:

- Add the proximity sensors A (105) and B (106) to the setup (Fig. 10.11).
- State that the train should behave as in Example 1.

*This example supplied by Dr. Ryan Durante.

*This example supplied by Dr. Ryan Durante.

Copyright © 2005 by Marcel Dekker

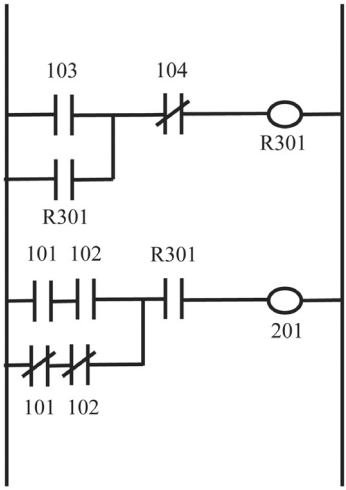


FIG. 10.10 PLC solution for train example 1

- But it should stop if it has crossed sensor A four times without crossing sensor B or crossed sensor B four times without crossing sensor A.

On the surface, one might ask:

- Why are we doing this?
- Does this not make the human operator a slave?

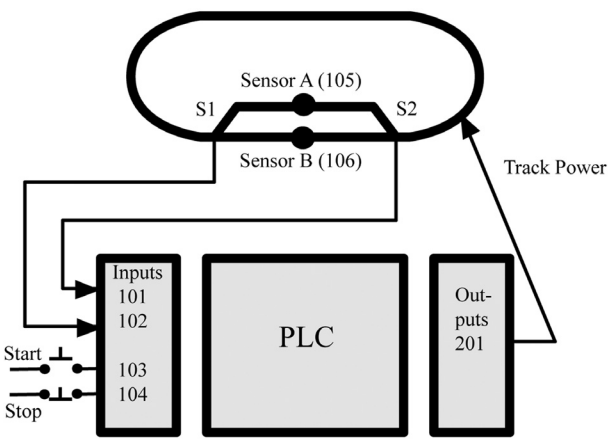


FIG. 10.11 Model train example 2

Copyright © 2005 by Marcel Dekker

We are doing this to create the need for counters, and to show the next level of difficulty to writing Ladder Diagrams. And yes, the human has a specific set of tasks to count how many times the train has crossed each loop, or the train will cease to run. In real life one would surely never want to implement this problem! It would be better to have the PLC operate the switches and take the human out of the loop (Fig. 10.12).

So we will need two counters and some elements to compare a counter to a fixed target number. The Ladder Diagram starts to become quite a bit more complex, and this example is only two loops of a model train. The rungs from the top are:

- Top two rungs to energize system relay R301.
- Third rung increments the counter C10 when sensor A is triggered.
- Fourth rung increments the counter C11 when sensor B is triggered.
- Fifth rung resets/enables the counter C10 when system is powered up or the other loop is transversed.
- Sixth rung resets/enables the counter C11 when system is powered up or the other loop is transversed.
- Seventh and eighth rungs compare counters C10 and C11 to power relay R302 when both true.
- Ninth and tenth rungs check for the switches S1 and S2 and allow power if relay R301 is energized and relay R302 is not energized.

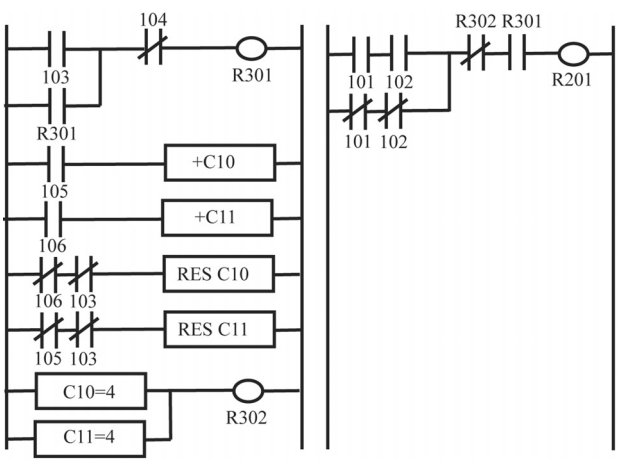


FIG. 10.12 PLC solution for train example 2

Copyright © 2005 by Marcel Dekker

10.3. OTHER PROGRAMMING OPTIONS

As one can see from the simple model train examples, the Ladder Logic can become complex very quickly with only a handful of inputs and outputs. Writing the Ladder Diagram on paper can be cumbersome. So there have been other methods developed to set up the logical relationships:

- Function Block Diagrams (FBD): Program elements appear as blocks, which get wired together in a manner resembling a circuit diagram. Blocks contain algorithms and wires indicate data flow. Programs like National Instrument's LabView fall into this category.
- Sequential Function Charts (SFC): Uses flow charts to develop the logical progression and different paths.
- Structured Text (ST): This method uses a high-level language (if-then-else, Case of, etc.). Examples would be C++ and Delphi.

Other directions are based on the existing knowledge of PLC programmers. One uses a PC interface to program the PLC (Fig. 10.13). A second option is Allen Bradley's SoftLogix 5, a Windows NT based soft control

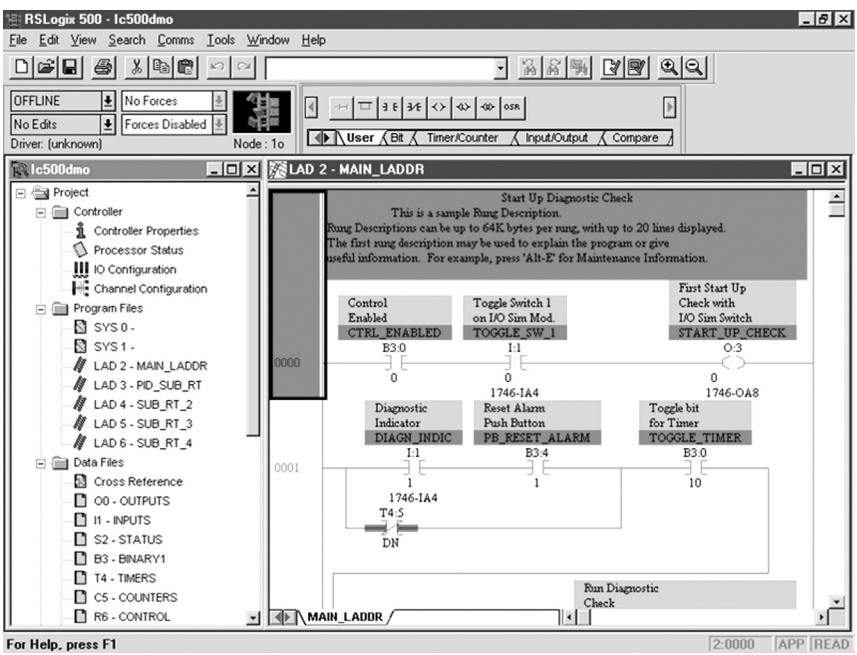


FIG. 10.13 RSLogix 500 programming environment (from Durante, 1999)

Copyright © 2005 by Marcel Dekker

engine that offers the openness and power of PC-based control, combined with the functionality of the PLC-5 processor. It also provides hardware and software compatibility with Allen-Bradley I/O, communications and Operator Interface devices. The SoftLogix 5 Controller is ideal for applications with large memory requirements or the need for a high level of information integration.

10.4. CASE STUDY NUMBER 1: AGILE AUTOMATION CONTROL SYSTEMS — THE HANSFORD ASSEMBLY FLEX PROJECT*

Flexible Automation needs more than a robot to be functional. The supporting feeders and conveying system must change quickly, as well as the software that coordinates the control. This is not a simple task if a company plans on creating a commercial product that will service the needs of a range of customers. Hansford Automation, Rochester, NY, made one of the early commercial attempts in the late 1990s.

This case study reviews the Hansford Assembly Flex project conducted at RPI (Durante, 1999). Assembly Flex is a modular automation assembly system in the truest sense. In most cases it can be reconfigured (re-tooled) in less than 20 minutes and go from building water pumps to car stereos. Much of this flexibility stems from its unique software control system developed by Dr. Ryan Durante, the author, and others.

The New York State Center for Automation Technologies (CAT) at Rensselaer Polytechnic Institute (RPI) was contacted by Hansford Automation Systems, for the purposes of entering into a cooperative effort to assist Hansford in the development. The CAT's role in the project was to assist Hansford in developing initial functional specifications and the corresponding software for the Assembly Flex system. Hansford had already determined the mechanical hardware configuration of the system. Figure 10.14 shows the three assembly modules (SCARA, Cartesian, and a Pick and Place) assembled to form a small demonstration system.

The mechanical concept was to have assembly modules (Fig. 10.15) that could accept up to six docking process modules (Fig. 10.16) to form a workcell. The number of process modules was limited to six because of the experience gained from the National Center for Manufacturing Sciences (NCMS) Light Flexible Mechanical Assembly (LFMA) project on which Hansford was the prime integrator. More than six stations per workcell was

*This example supplied by Dr. Ryan Durante.

Copyright © 2005 by Marcel Dekker

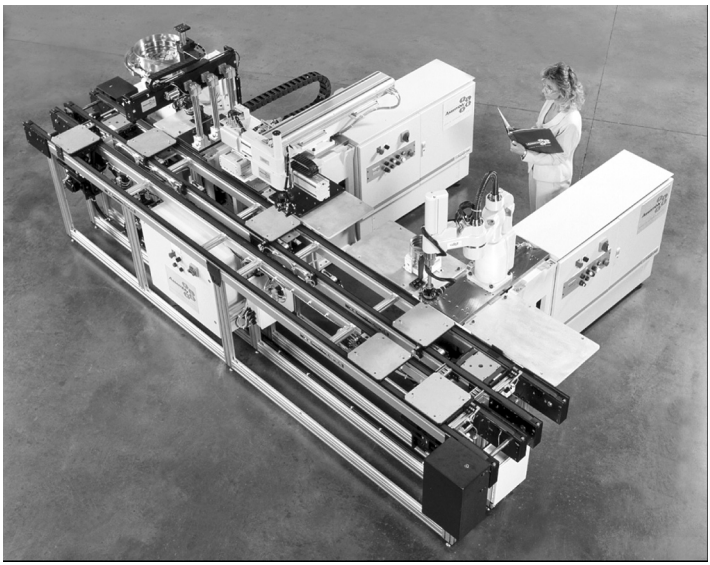


FIG. 10.14 Three cell Assembly Flex system (from Durante, 1999)

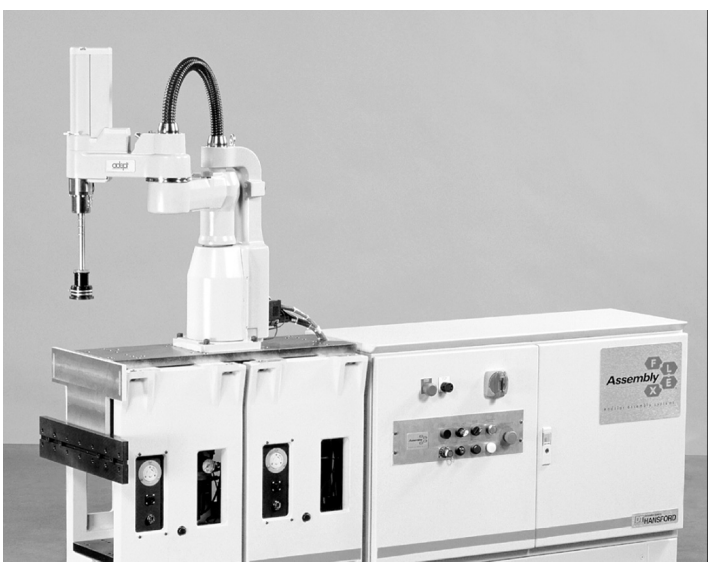


FIG. 10.15 Single SCARA Assembly Flex assembly cell (from Durante, 1999)

Copyright © 2005 by Marcel Dekker

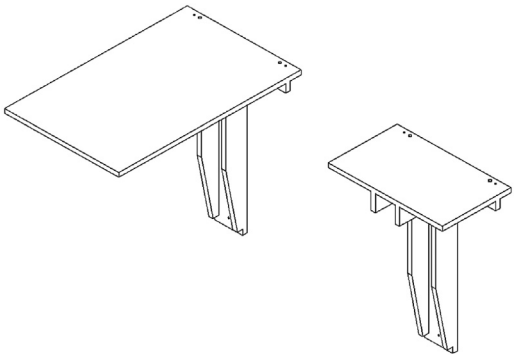


FIG. 10.16 Long and Short Process Modules (from Durante, 1999)

found to be too complicated and caused too much congestion (Molnar and Rezsonya, 1997).

The process modules mount via a three-point kinematic mounting mechanism, which provides highly accurate registration and repeatability. Hansford knew that they wanted the system's mechanical design and controller architecture to be open and modular. They also realized that this would not happen overnight and accepted that the first generation of the Assembly Flex system would be Adept-centric, that is, the robot assembly cells would use Adept MV controllers. They also required that all device-level communication would be via DeviceNet — an up and coming (in the late 1990s) open standard for field (device) level communication.

10.4.1. Agile Control System: Release 1.0

The first official release of the software had to meet the following specifications:

1. All screens must be active.
2. Configuration was to be channel based (described in Durante, 1999).
3. The following modules were supported:
   - SCARA Assembly;
   - Cartesian Assembly;
   - Pick and Place Assembly;
   - Bowl Feeder;
   - Tray Stacker;
   - Intelligent Interface Module (IIM).

The final Assembly Flex Control Suite is composed of three applications: the System Executive, the Event Viewer, and the Configuration Editor.

Copyright © 2005 by Marcel Dekker

System Executive

Figure 10.17 shows the main screen of the System Executive. The screen shot shows a system navigator on the left side of the screen. The system executive uses the multiple document interface (MDI) for displaying cell information while maximizing screen real-estate. Four cells are tiled in the client area. The screen is divided into five key sections: client area, cycle control, system navigator, status bar, and event viewer. Depending on the level of security given to any user, they may be able to change the system configuration and/or the robot program, or they may be limited to simply observing the data and permitted to only hit the system stop button for safety reasons.

Figure 10.17 has four module inspectors displayed. The MDI interface allows for displaying multiple module information simultaneously while guaranteeing that other salient features of the human machine interface (HMI) are never obscured from the operators view, particularly the Cycle Control and Event Viewer. It also allows the user to tile and cascade the windows automatically.

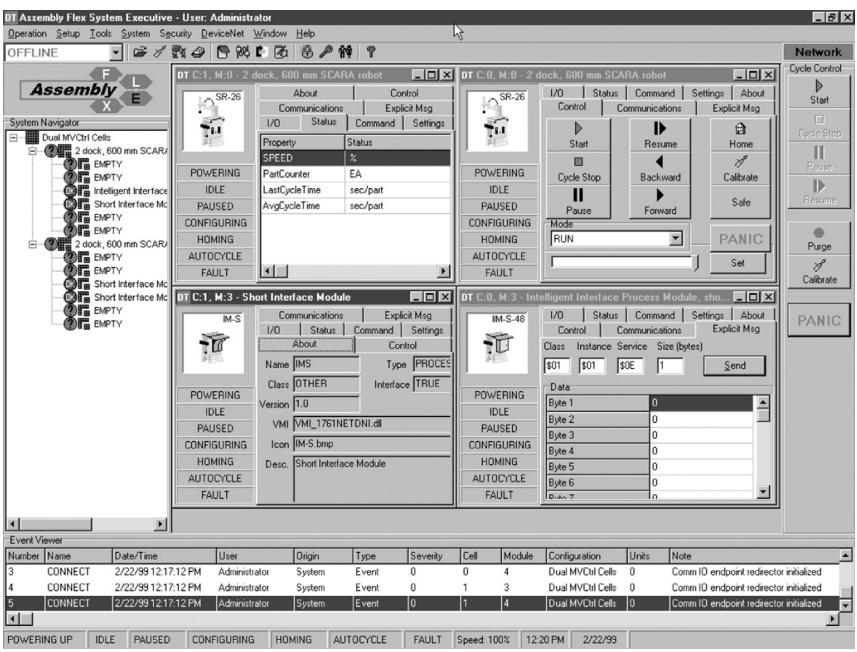


FIG. 10.17 System Executive main screen

Copyright © 2005 by Marcel Dekker

A module inspector exists for every module defined in the system. The module inspector is the user's interface into a module. It allows control at all levels, data monitoring, and atomic-level control of all discrete I/O. It displays state information, and allows the user to send explicit messages to the target module. All I/O points are tagged here and displayed. The user can toggle all discrete I/O from this single screen.

The Cycle Control provides system-level supervisory control. It provides Cycle Start, Cycle Stop, Pause, Resume, Purge, Calibrate and Panic Stop functionality to the user. A Cycle Start sends the start command to all workcells to initiate the build cycle. A Cycle Stop will instruct all workcells to stop after their current cycle is complete. This type of stop leaves the system in a known state. This can be contrasted to the Panic Stop, which will force the system to shut down as rapidly as possible. In this case, the system is left in an unknown state and must be calibrated before beginning the next build cycle. The Pause button puts the system into the "pausing on" state. This allows the user to single step through each assembly step for debugging and pilot runs. Resume returns the system to the autocycle state. Purge instructs all cells in the system to completely purge all products from their workspace. This would most likely be done after a Panic Stop.

Event Viewer

The Event Viewer provides the user with a log of all events from the current session including the event number, name, date/time stamp, the user, the origin, the type, severity, the source module, the configuration, the units, and a generic text notation. This provides the user with a quick look at recent events without opening the Event Viewer application.

Configuration Editor

The Configuration Editor is a design time tool that allows drag and drop configuration of the target system. The main screen is shown in Fig. 10.18. The design window is broken up into four major sections: the system information box, the module palette, the module information box, and the workcell design box.

The system information box gives the user system-level information such as name, author, creation date, revision, and description. This information corresponds to what is presented in the system information window of the System Executive.

The module palette utilizes a two-level tabbed worksheet. The top level splits the available modules into assembly (base) and process modules. The assembly modules are then broken down into SCARA, Cartesian, Pick and Place, Conveyor, and Other. Process modules are broken down into feeder, handler, and other (Fig. 10.19).

Copyright © 2005 by Marcel Dekker

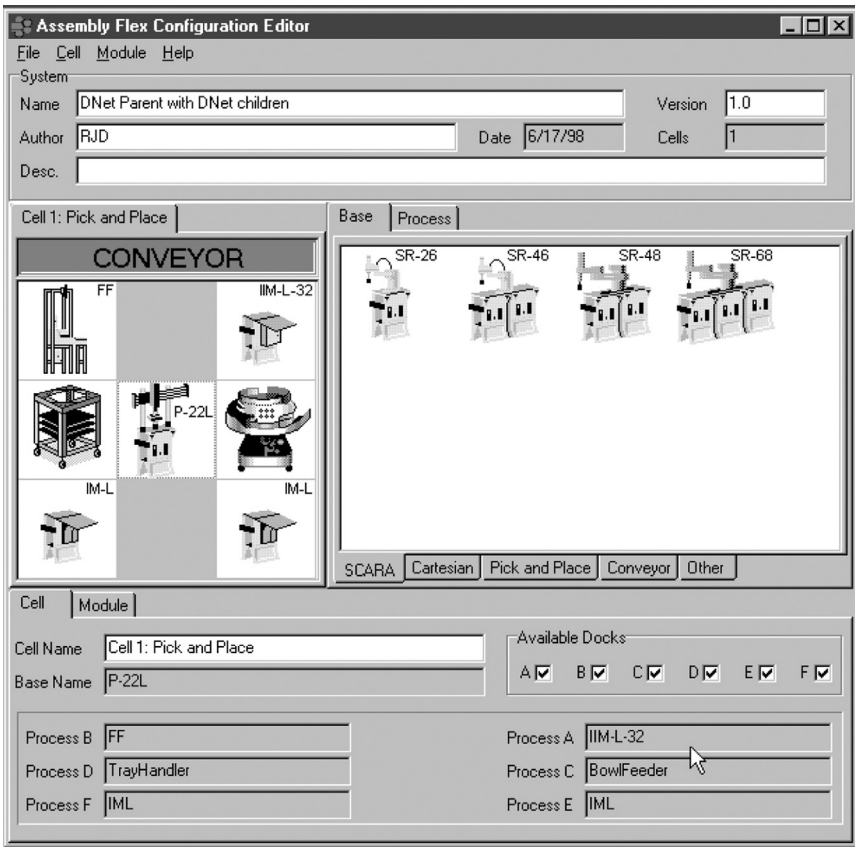


FIG. 10.18 Configuration Editor

The workcell design box is where the target system is constructed. Figure 10.20 shows a two-cell system with a SCARA-based workcell displayed. This workcell shows three active process modules, a tray feeder, a bowl feeder, and an intelligent interface module. The box is tabbed for each cell in the system. Assembly modules can be dropped into the center cell and process modules connected to it. All communication links are created automatically. This corresponds to the physical placing of process modules on either side of the robot, and the proper connection of the standardized cabling. The user no longer has to be concerned about the software logical assignments. The software has performed that task for the user.

Copyright © 2005 by Marcel Dekker

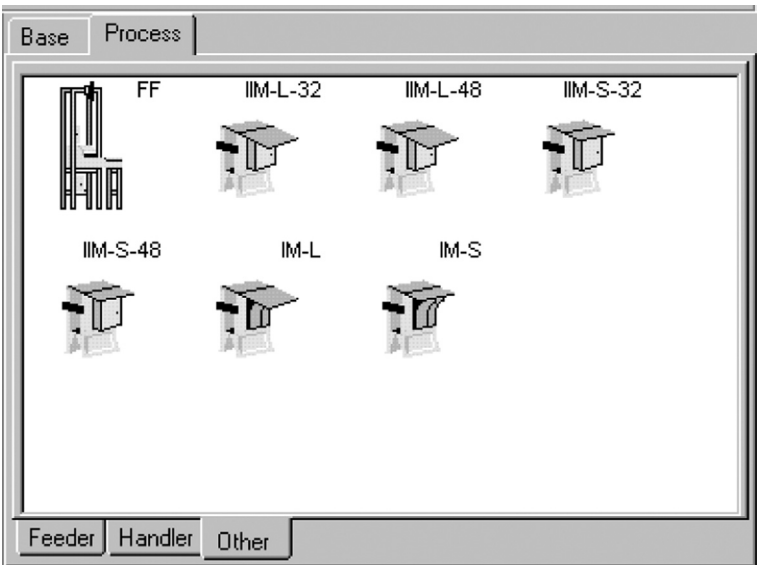


FIG. 10.19 Module Palette

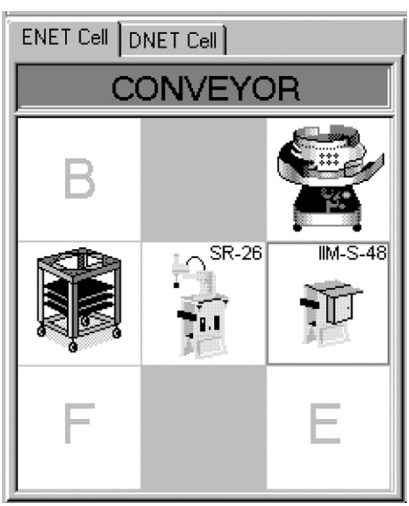


FIG. 10.20 Workcell Design Box

Copyright © 2005 by Marcel Dekker

10.4.2. Control System Architecture

The first prototype of this architecture was demonstrated at the 1997 Assembly Technology Exposition (ATE) show in Chicago. It had to meet the following requirements:

1. The Ethernet/DDE link had to be in place and functional (to support the following two tasks).
2. The System Executive had to be able to start/stop the system.
3. Main displays had to include "live feedback" including robot status, PLC information, and conveyor status.
4. All other screens were to be present but not functional.

Figure 10.21 shows the DeviceNet communications scheme in more detail.

This was not the ideal configuration but was driven by the technology available at the time. When the new DeviceNet scanner cards became available, it was possible to have a direct physical connection between DeviceNet and the PC. This did away with the SLC 504 PLC/scanner card and the RS232 serial link. Also, since Allen Bradley has recently released the new Micrologix PLCs, it is now possible to do away with the DNDB CAN interface module. The configuration that will be implemented in the final architecture is shown in Fig. 10.22.

Figure 10.23 shows the finalized robot communications scheme in more detail. The prototype that was developed enjoyed good acceptance at the ATE show. After taking into considerations suggestions and criticisms from the show, work began on the analysis and design of the "real" software architecture.

10.4.3. A Second Assembly Flex Implementation

The power of Assembly Flex as a control software architecture was not fully appreciated until it was implemented on a second automation configuration. A linear track based tray handling robot (Fig. 10.24) had been built by American Dixie Group, but the controller was not implemented. The tray handling robot was outfitted with Assembly Flex by Dr. Ryan Durante in a matter of weeks.

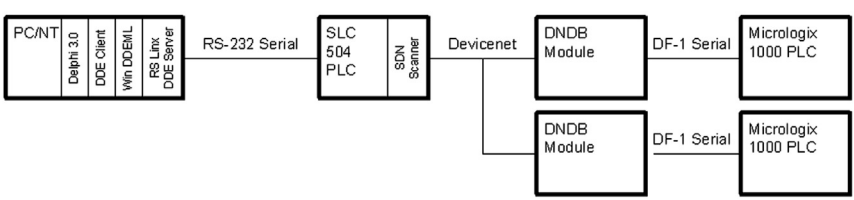


FIG. 10.21 DeviceNet system used for prototype

Copyright © 2005 by Marcel Dekker

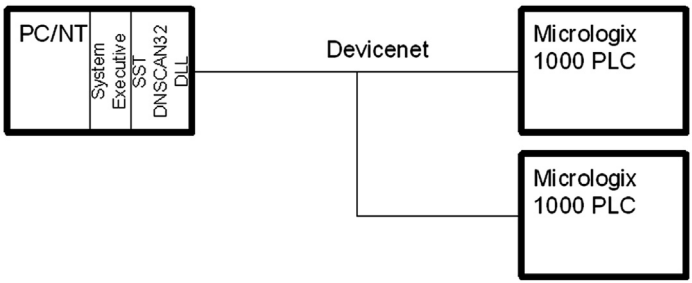


FIG. 10.22 Ideal DeviceNet configuration

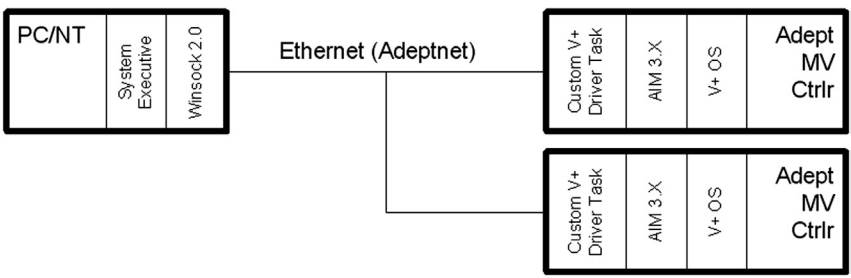


FIG. 10.23 Robot communications implementation

The actual robot (Fig. 10.25) shows a gripper with eight large vacuum-powered suction cups (located directly above the roller conveyor) on a dual arm implementation. There are a tremendous number of vacuum and pneumatic hoses, sensors and electrical wires around the vertical arm. This system was truly a prototype. The mail containers are not shown in this figure for clarity.

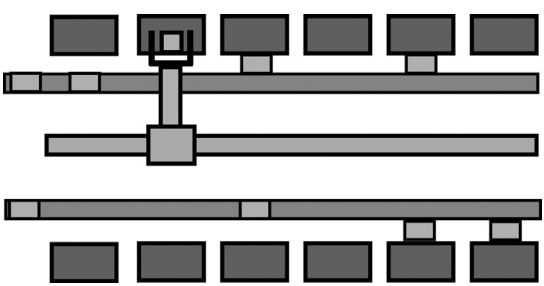


FIG. 10.24 Linear track based single arm tray handling robot concept

Copyright © 2005 by Marcel Dekker

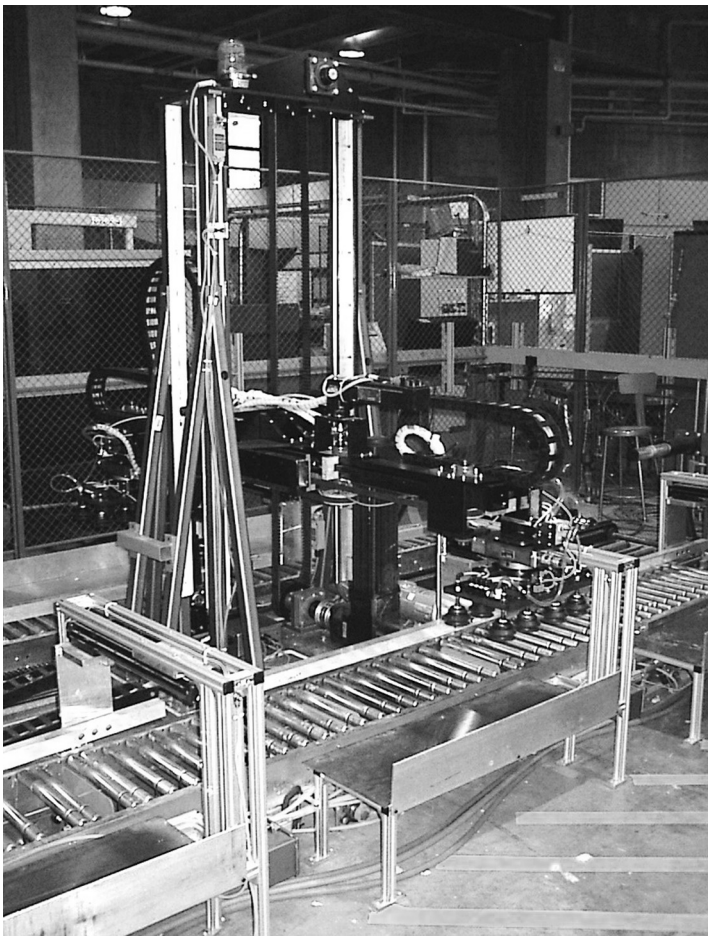


FIG. 10.25 Dual arm track based tray handling robot

The controller box was 5 ft high by 6 ft wide (Fig. 10.26), and the electrical power being used was significant.

The implementation of Assembly Flex was straightforward. A customized Human Machine Interface computer display was designed (Fig. 10.27) to represent the robot and the mail container locations. The underlying drivers for the specific electronic motion controllers were updated from the original implementation, similar to the updating of drivers for your PC when you purchase a new model printer.

Copyright © 2005 by Marcel Dekker

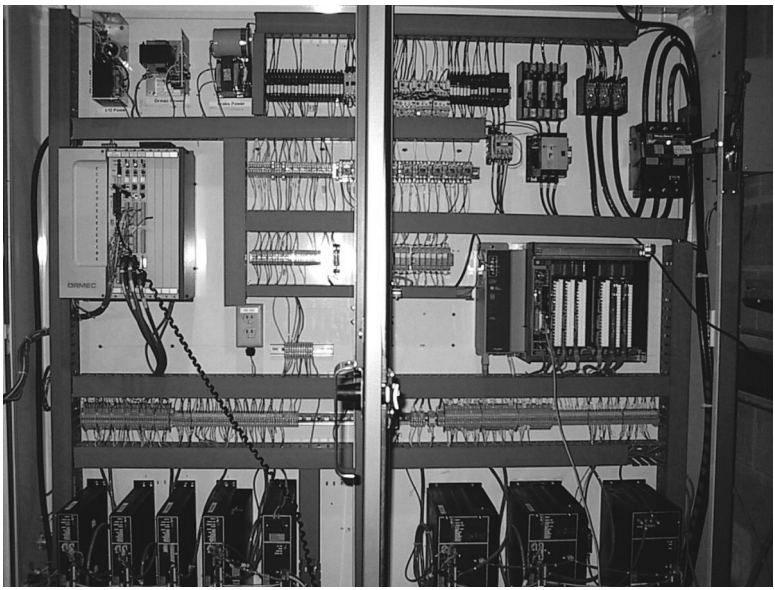


FIG. 10.26 Controller for dual arm track based robot

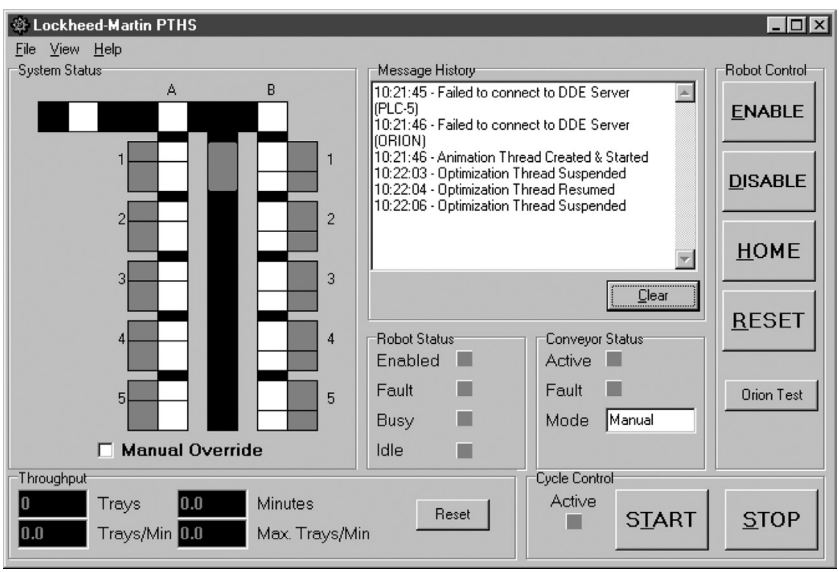


FIG. 10.27 Assembly Flex implemented for dual arm tray handling robot

Copyright © 2005 by Marcel Dekker

10.4.4. Project Conclusions

The Hansford Assembly Flex system was shown at the 1998 ATE as well as the IMTS (International Manufacturing Trade Show). The control was accomplished by a fully functional agile automation control system. Final cut over of the software base and documentation was done in December 1998. The straightforward implementation for the tray handling robot application demonstrated the usefulness of modular software.

This system has demonstrated the successful use of Agile Automation Control Systems in flexible automation. Users of all levels of security clearance can use a drag and drop interface with the appropriate levels of system information and control to quickly change over a flexible workcell. The object-oriented approach will allow for quick implementation of new robots and workcell elements.

As the first flexible automation commercial entry into the field, the Assembly Flex system was met with some resistance. Consumers were not accustomed to thinking in terms of flexibility and its longer term financial benefits. As the economy for automation cycled through lean times, Hansford Automation was purchased by DT Industries, and the Assembly Flex project tabled as personnel were dispersed. The next generation will learn from the successes and failures of Assembly Flex in terms of its market and customer base.

10.5. CASE STUDY NUMBER 2: OMAC AUTOMATION CONTROL

A large effort to have improved automation machine control is being conducted by the Open Modular Architecture Controls (OMAC) organization. Started in the 1980s by General Motors, factory floor technical managers were finding that each and every machine from multiple vendors had unique controllers and/or software implementation. Technicians were going to training classes for each machine, and the retained knowledge when a worker moved to a new site or a new company was too limited. So the OMAC effort was initiated. It grew into the packaging automation area in the late 1990s and is flourishing. It is likely that many of the big and medium customers will be insisting on OMAC compatible controllers in the near future, so this can impact on much of the automation industry.

OMAC's packaging working group's Mission Statement is to "enhance the value of packaging machinery and systems by promoting the use of digital motion control and OMAC guidelines for open control architectures." The stated goals (OMAC, 2000) in 2000 were, over a three-year horizon and with no increase in capital or operating cost, potential improvements that should approach 50% for:

Copyright © 2005 by Marcel Dekker

- Delivery lead time;
- Startup time;
- Machine/system footprint;
- Product throughput;
- Product changeover time;
- Material loss;
- Machine reconfiguration/overhaul time;
- Machine downtime;
- Mean time between failure.

These goals are very aggressive, and many people in the automation industry (including the author) wonder aloud if they are all possible, but they are worthy goals and should be pursued. Giant customers like Hershey's (Campbell, 2000) see that these goals are needed to stay cost competitive, and that some new packaging machines are already making headway. They see:

- Smaller footprints;
- Faster startups;
- Flat to lower cost per function completed;
- Higher speeds;
- Improved efficiency;
- Faster changeovers;
- Better quality package;
- Reduced waste;
- Improved reliability.

Hershey's also sees that redesigning packaging machines with a clean sheet of paper using Mechatronics can make a big impact:

- Fewer moving parts;
- More precise movements;
- Software is major performance and cost component;
- Machines are more modular;
- Functions added by module/software;
- Fewer machine platforms;
- Performance changes within a single machine platform by changing controls/software.

10.5.1. OMAC Working Groups

There are a variety of subcommittees within the OMAC Packaging Workgroup effort. These subcommittees have changed over time to dynamically mold into the best methods for success, and will likely continue to change in the near future. So the specifics of the various subcommittees will not be

Copyright © 2005 by Marcel Dekker

detailed here. The reader is encouraged to check their website for the latest results.

However, there are a few nuggets of wisdom that will most likely not change over time. The fact that most automation machine control programs can be written a thousand different ways, having someone other than the original code writer try to understand someone else's code can take forever. There are basics that need to be agreed upon so as to create some sanity in the situation.

So independent of the type of controller, the language used, or any specific brand, one of the goals of the PackML subcommittee of the OMAC Packaging Workgroup is to define two classes of mode of operation:

- User Selectable: Two types of Mode exist within the User Selectable Class: Procedural (in which there are three possible modes of operation: Automatic, Semi-Automatic and Manual) and Equipment Entity (in which there are two possible modes of operation: Automatic and Manual).
- Machine: Machine modes provide the control framework in which the User Selectable modes become available for selection. These can be transparent to the operator. The two proposed modes are ESTOP and IDLE.

The user selectable modes are selected by the machine operator in order to perform relevant procedures and operations (Fig. 10.28). These are similar to most robotic controllers and many automation machines, but some manufacturers have in the past created their own branching and unique definitions that can be confusing to the majority of the workforce, and sometimes even a safety risk, so standards like this are not universally accepted by default.

Machine modes provide the control framework in which the User Selectable modes become available for selection (Fig. 10.29). This concept comes from the need for engineers determining the control sequences to define the

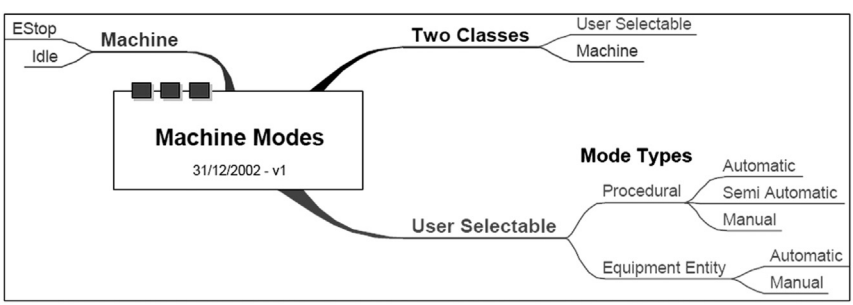


FIG. 10.28 OMAC machine modes (from OMAC PackML Team, 2003)

Copyright © 2005 by Marcel Dekker

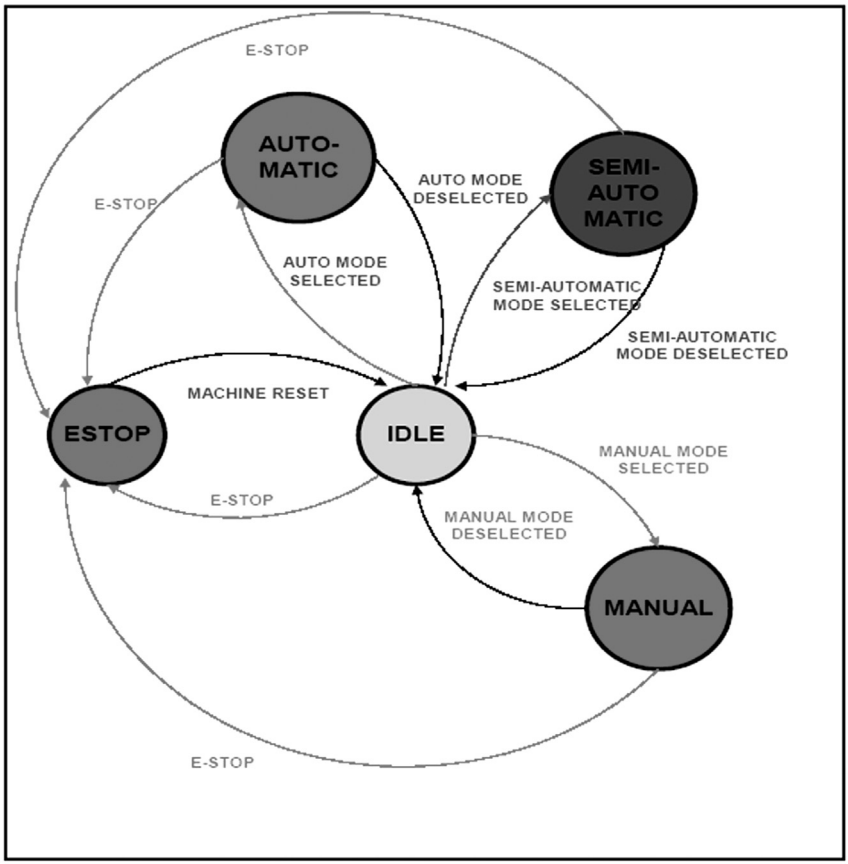


FIG. 10.29 OMAC machine state diagram (from OMAC Pack ML Team, 2003)

possible states the machine can be in at any time. Another way of approaching this issue is to consider at any point in the automation machine's operation, what are the conditions of the following:

- sensors;
- actuators;
- moving members;
- incoming parts;
- processed parts;
- facility supplied pneumatic lines and electrical power.

Copyright © 2005 by Marcel Dekker

If any of these fail, due to loose connections, power variations, or the unanticipated (a stray animal decides to nest in your machine for the winter), what happens to your machine control? Does it totally hang up and freeze? Does it send the motors running at full pace and create an internal crash? Do you have to reboot like one does sometimes on a PC?

It is difficult to anticipate all of life's possibilities on your machine going wrong, but many of these are predictable. So if they happen, what state does the machine go into? Or will it just hang while running, which can be a safety nightmare. Figure 10.29 gives a framework to use while sorting out these issues, which is consistent with many of the other new machines being built today.

10.5.2. Conclusion: Do You Use OMAC Principles?

Depending on your customer base, you may not have a choice. Over 100 big customers are on the list of OMAC users that will require compliant automation controllers, or they have promised to buy your competition's machine, even if it costs 10–25% more. The OMAC compliant controller will save them time and money in the long run.

10.6. CONCLUSIONS

Controllers have become almost easy to use. Any engineer, no matter what their background, can learn a specific type of control, whether Ladder Logic, PC based, or microcontroller. The competition within suppliers and the consistency demanded by OMAC will help everyone win.

PROBLEMS

1. Modify Model Train Example 1 to operate the system if one has added two actuators to replace the human intervention for switches S1 and S2, and create the Ladder Diagram.
2. Create a Timing Diagram for making a submarine sandwich (roll, meat, lettuce, cheese, sauce, etc.). What is the impact of changing the order of operations?
3. Analyze eating an ice cream sundae and draw a block diagram of what you do and under what conditions. What options do you have and what are the impacts?
4. Plan a party for you and some friends, but model it with a Timing Diagram. What are the inputs and outputs?

Copyright © 2005 by Marcel Dekker

PROJECT ASSIGNMENT

1. Write a list of all inputs and outputs for your project.
2. Draw a Timing Diagram of your machine's normal operational mode.
3. Create a Ladder Diagram.
4. Specify a PLC that would work for controlling your machine. Compare to a PC-controlled system.

REFERENCES

Campbell, K. (2000). OMAC at Hershey Foods Corporation. 2000 Pack Expo Conference, November 3–8, Chicago, IL.

Durante, R. (1999). A Design Methodology for Developing Agile Automation Control Systems Architectures. PhD thesis, Rensselaer Polytechnic Institute, May.

Molnar, M., Rezsonya, T. (1997). Results Summary Report for the NCMS Light Flexible Mechanical Assembly Program. NCMS Internal Memo, May 9.

OMAC (2000). Open Modular Architecture Control for Packaging Machinery — A User's Perspective. Pack Expo Solutions 2000 Conference, November 7.

OMAC Pack ML Team. (2003). *Machine Mode Definition V0.3*. www.omac.org.

Copyright © 2005 by Marcel Dekker